

Object-oriented Programming 2 KU

gRPC Introduction

Note: This is NOT relevant anymore for Assignment 0

Sebastian David Albu

Institute of Computer Graphics and Knowledge Visualization (CGV)

Code on slides generated using [Carbon](#)

Assignment 1

Now the fun begins ;)

Example: EventServer Java and gRPC + proto

Event Service in gRPC + Java



- Follow along if you want again 😊
 - Download and open [this project here from TC](#)
 - The provided example shows how a gRPC server can handle requests and a client can interact with the server using generated stubs.



What is gRPC?

Introduction to gRPC



- gRPC (gRPC Remote Procedure Calls) is a high-performance, open-source RPC (Remote Procedure Call) framework developed by Google.
- It allows services to communicate across different environments efficiently, using Protocol Buffers (protobuf) as the interface definition language.
- gRPC works across various platforms and languages (e.g., Java, Python, Go).
- Enables low-latency, high-throughput communication between services.
- Uses proto files to generate client and server stubs, reducing the need for manually writing boilerplate code.

gRPC-Workflow



- Request-Response Model:
 - Clients call a remote procedure (like calling a method on an object) and await the response.
- Uses HTTP/2:
 - Supports streaming, multiplexing, and other HTTP/2 features for better performance.
- Binary Serialization:
 - Messages are serialized using Protocol Buffers for faster communication.

gRPC-Workflow



- Client and Server:
 - Clients make requests, and servers process them.
- Proto Files:
 - Define the structure of requests, responses, and services.
- Stub:
 - A client-side object that allows remote method calls as if they were local.

What is proto?

The importance of proto files



- What is a Proto File?
 - A file that defines the structure of gRPC messages and services using Protocol Buffers.
- Why is it Important?
 - Provides language-neutral and platform-neutral service definitions.
 - Enables auto-generation of code for clients and servers in different languages.

The importance of proto files



```
syntax = "proto3";
import "google/protobuf/empty.proto";
package eventservice;
option java_package = "at.tugraz";
option java_multiple_files = true;
option java_outer_classname = "EventServiceProto";
option java_generic_services = true;

message Event{
    int64 id = 1;
    string message = 2;
    int64 timestamp = 3;
}
message Events{
    repeated Event events = 1;
}
message EventCreation{
    string message = 1;
}
message EventById{
    int64 id = 1;
}
service EventService{
    rpc events(google.protobuf.Empty) returns (Events){};
    rpc eventById(EventById) returns (Event){};
    rpc logEvent(EventCreation) returns (Event){};
}
```

Understanding proto messages



- Message:
 - A message defines the structure of the data exchanged between the client and the server.
 - Example:

```
message Event {  
  int64 id = 1;  
  string message = 2;  
  int64 timestamp = 3;  
}
```

Understanding proto services



- Service:
 - Describes the methods available for remote procedure calls.
 - Example:

```
service EventService {  
  rpc events(google.protobuf.Empty) returns (Events);  
  rpc eventId(EventById) returns (Event);  
  rpc logEvent(EventCreation) returns (Event);  
}
```

```
service EventService {  
  rpc events(google.protobuf.Empty) returns (Events);  
  rpc eventById(EventById) returns (Event);  
  rpc logEvent(EventCreation) returns (Event);  
}
```

- `rpc events(google.protobuf.Empty):`
 - Method `events` takes an empty request and returns a list of events (`Events`).
- `rpc eventById(EventById):`
 - Method `eventById` takes an ID and returns the corresponding event.
- `rpc logEvent(EventCreation):`
 - Method `logEvent` takes an event creation request and returns the created event.

Java + gRPC Server-Side

Java Server



- Java Server:
 - Implements the gRPC service defined in the proto file.
 - Uses ServerBuilder to create and start the server.

```
Server server = ServerBuilder.forPort(6969)
    .addService(new EventServiceImpl())
    .build();
```

- Java Client:
 - Uses a stub generated from the proto file to communicate with the gRPC server.

Java Server



- Java Server Setup:
 - Creates a server on port 6969 and adds the EventServiceImpl to handle requests.

```
try
{
    server.start();
    logger.info(String.format("Server has started on port %d", port));
    server.awaitTermination();
}

catch (Exception e)
{
    e.printStackTrace();
}
```

- Server Lifecycle:
 - Starts the server (server.start()) and waits for it to terminate (server.awaitTermination()).

EventServiceImpl



- What is this class?
 - It implements the gRPC service defined in the proto file by extending *EventServiceGrpc.EventServiceImplBase*.
 - Handles Requests:
 - Processes incoming gRPC requests for events, event by ID, and logging new events.

EventServiceImpl



- Handles Event Requests:
 - Responds with a list of all events. It uses StreamObserver to send the response back to the client.

```
@Override
public void events(Empty request, StreamObserver<Events> responseObserver)
{
    Events.Builder eventsBuilder = Events.newBuilder();
    eventsBuilder.addAllEvents(events);
    Events allEvents = eventsBuilder.build();

    responseObserver.onNext(allEvents);
    responseObserver.onCompleted();
}
```

EventServiceImpl



- Handles EventById Requests:
 - Finds an event by its ID from the stored list of events and sends it back to the client.

```
@Override
public void eventById(EventById request, StreamObserver<Event> responseObserver)
{
    Event event = events
        .stream()
        .filter(e -> e.getId() == request.getId())
        .findAny()
        .orElse(null);

    responseObserver.onNext(event);
    responseObserver.onCompleted();
}
```

EventServiceImpl



- Handles LogEvent Requests:
 - Receives an event creation request, creates a new event, and adds it to the list.
 - Responds with the newly created event.

```
@Override
public void logEvent(EventCreation request, StreamObserver<Event> responseObserver)
{
    String message = request.getMessage();
    Event event = Event.newBuilder().setId(id++).setMessage(message).
                                setTimestamp(new Date().getTime()).build();
    events.add(event);

    responseObserver.onNext(event);
    responseObserver.onCompleted();
}
```

Java + gRPC Client-Side

EventPlayground



- Client-Side Code:
 - Uses ManagedChannel to connect to the server running on localhost:6969.
 - EventServiceGrpc.EventServiceBlockingStub enables the client to make blocking RPC calls.

```
public static void main(String[] args)
{
    ManagedChannel channel = ManagedChannelBuilder.forAddress("localhost", 6969)
        .usePlaintext()
        .build();

    EventServiceGrpc.EventServiceBlockingStub stub = EventServiceGrpc.newBlockingStub(channel);
```

Client Requests



- Log Event Request:
 - Sends a message to the server, creates an event, and logs it as a new event.

```
EventCreation request1 = EventCreation.newBuilder().setMessage("00P2").build();  
Event response1 = stub.logEvent(request1);
```

- EventById Requests:
 - Requests an event by its ID.

```
EventById request3 = EventById.newBuilder().setId(1).build();  
Event response3 = stub.eventById(request3);
```

Proto File in Action

Proto File



- Generating Code:
 - The proto file defines the structure for both client and server.
 - Tools generate stubs for easy RPC communication.
- How It Connects:
 - gRPC uses the proto file to ensure consistent communication between client and server.

Conclusion and Further Resources

Conclusion



- gRPC is efficient, scalable, and cross-platform.
- Proto files are essential for defining the structure of gRPC services.
- The provided example shows how a gRPC server can handle requests and a client can interact with the server using generated stubs.

Further Resources



- [What is gRPC?](#)
- [Introduction to gRPC \(Baeldung\)](#)
- [gRPC with Insomnia](#)
- [The sample code used today \(TC\)](#)
- [Tutorial from last year \(same code\) – Tube recording](#)
 - (You can skip ahead – JTS will be covered in the next week(s))
 - (Check out Discord #tutorials for more resources !!)

Object-oriented Programming 2 KU

gRPC Introduction

Sebastian David Albu

Institute of Computer Graphics and Knowledge Visualization (CGV)

Code on slides generated using [Carbon](#)